JS_Day8_Hands On-Chintha Ramkrishna

Problem1 :-

How can we submit feedback using a button click and display a confirmation message dynamically without refreshing the page?

Code :-

```
<!--Requirements

• Create a webpage with a "Submit Feedback" button.

• When clicked, display a confirmation message dynamically.

• Use JavaScript event listeners to handle the click event.



 Technical Constraints

• Use plain HTML, CSS, and JavaScript.

• Event handling must use addEventListener().

• No external frameworks allowed.



Expectations:

• Proper DOM manipulation.

• Clean and readable code.-->




<!DOCTYPE html>

<html>

<head>

<title>Feedback Validation</title>
```

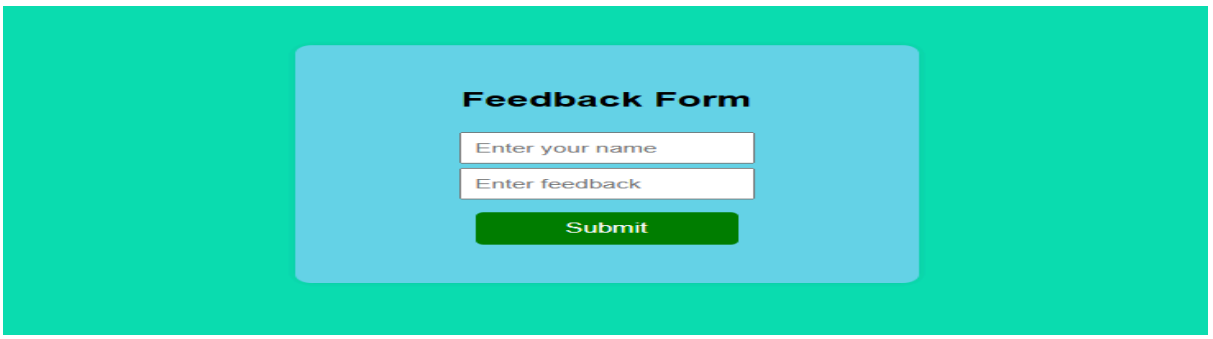
```
<style>
body{
    font-family:Arial;
    background:rgb(14, 224, 179);
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}

.container{
    background:rgb(100, 213, 233);
    padding:30px;
    width:320px;
    border-radius:10px;
    box-shadow:0 0 10px rgba(0,0,0,0.1);
    text-align:center;
}

input{
    padding:8px;
    width:50%;
    font-size:15px;
    margin-top:5px;
```

```
}

button{
    padding:10px;
    font-size:16px;
    margin-top:15px;
    background:green;
    color:white;
    border:none;
    border-radius:5px;
    cursor:pointer;
    width:50%;
}

#message{
    margin-top:15px;
    font-size:16px;
}
</style>
</head>

<body>

<div class="container">
<!-- Heading for the feedback form -->
```

```
<h2>Feedback Form</h2>


<input type="text" id="nameText" placeholder="Enter your name">

<input type="text" id="feedbackText" placeholder="Enter feedback">

<!-- Submit button-->

<button id="submitBtn">Submit</button>


<div id="message"></div>


</div>


<script>

//Get references to DOM elements

const nameInput = document.getElementById("nameText");

//Get element for feedback input

const feedbackInput = document.getElementById("feedbackText");

//Get reference to the submit button and message display element

const btn = document.getElementById("submitBtn");

const msg = document.getElementById("message");

//Add click event listener to the button to handle feedback submission

btn.addEventListener("click", () => {


    const name = nameInput.value.trim();

    const feedback = feedbackInput.value.trim();

//Validate input and display appropriate message
```

```
    if(name === "" || feedback === ""){

        msg.textContent = "❌ Please enter name and feedback.";

        msg.style.color = "red";

    }

    else{

        msg.textContent = `✅ Thank you ${name}! Feedback submitted.`;

        msg.style.color = "green";


        nameInput.value = "";

        feedbackInput.value = "";

    }


});


</script>


</body>

</html>
```

Output :-



Explanation :-

- The webpage contains a button that allows users to submit feedback interactively.
- JavaScript uses addEventListener() to detect the button click event.
- When the button is clicked, DOM manipulation is used to display a confirmation message instantly.
- The page does not refresh because the interaction is handled entirely on the client side.
- This demonstrates basic browser event handling and dynamic UI updates using Vanilla JavaScript.

Problem 2 :-

Develop a Theme Toggle application using Vanilla JavaScript that allows users to switch between Light Mode and Dark Mode with a toggle button, dynamically changes background and text colors, stores the selected theme in localStorage, uses addEventListener for event handling without CSS frameworks, ensures smooth theme switching with clean modular functions, and demonstrates user interaction handling along with browser storage management.

Code :-

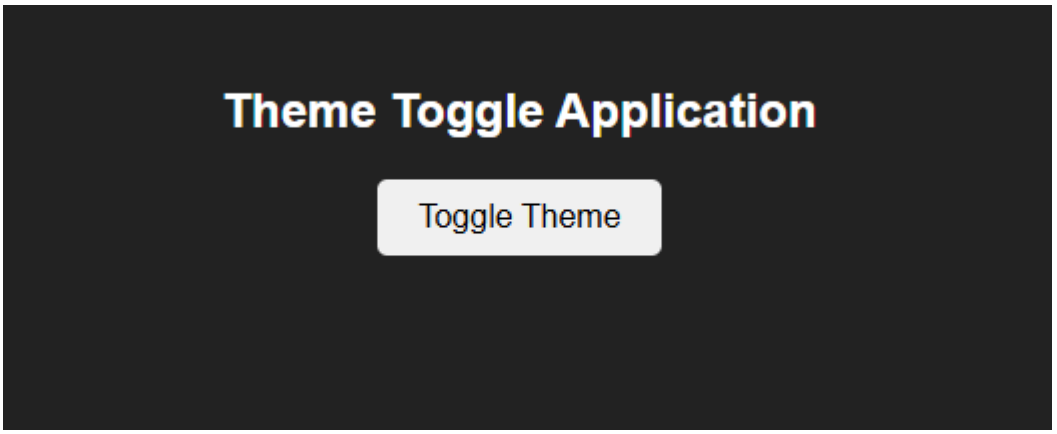
```
<!DOCTYPE html>

<html>

<head>

<title>Theme Toggle App</title>



<style>

body{

    font-family:Arial;
```

```
    transition:0.3s;
    text-align:center;
    padding-top:100px;
}

/* Light theme */
.light{
    background:white;
    color:black;
}

/* Dark theme */
.dark{
    background:#222;
    color:white;
}

button{
    padding:10px 20px;
    font-size:16px;
    border:none;
    border-radius:5px;
    cursor:pointer;
}
</style>
```

```
</head>


<body>


<h2>Theme Toggle Application</h2>

<button id="toggleBtn">Toggle Theme</button>


<script>

const body = document.body;

const btn = document.getElementById("toggleBtn");


/* Load saved theme */

const savedTheme = localStorage.getItem("theme");


if(savedTheme){

    body.classList.add(savedTheme);

}else{

    body.classList.add("light");

}


/* Toggle function */

function toggleTheme(){

    if(body.classList.contains("light")){

        body.classList.remove("light");

        body.classList.add("dark");
```

```
        localStorage.setItem("theme","dark");

    }else{

        body.classList.remove("dark");

        body.classList.add("light");

        localStorage.setItem("theme","light");

    }

}



btn.addEventListener("click",toggleTheme);

</script>



</body>

</html>
```

Output :-



Explanation :-

- This application allows users to switch between light and dark themes using a toggle button.

- CSS classes are used to change background and text colors dynamically.
- JavaScript listens for button click events using addEventListener().
- The selected theme is saved in localStorage so the preference remains after page reload.
- The code is modular by separating the toggle logic into a function.

Problem 3 :-

Develop an interactive To-Do List application using Vanilla JavaScript that allows users to dynamically add, delete, and mark tasks as complete without refreshing the page, uses DOM manipulation and event delegation for better performance, follows a modular code structure with reusable functions, provides a fully functional UI, and demonstrates advanced DOM event handling along with the concept of event delegation and the role of libraries in speeding up development.

Code :-

```
<!DOCTYPE html>

<html>

<head>

<title>Dynamic To-Do List</title>



<style>

body{

    font-family:Arial;

    background:#0a7275;

    display:flex;

    justify-content:center;
```

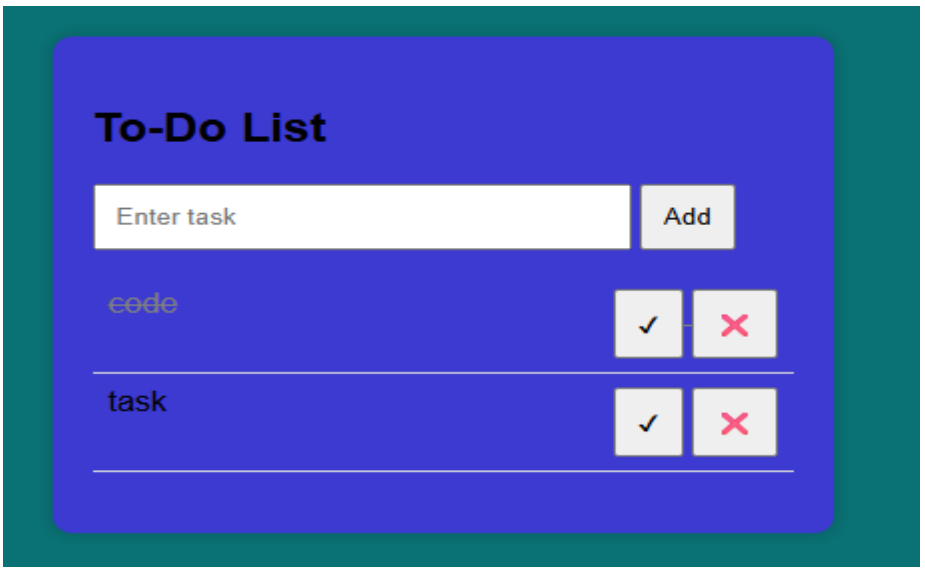
```
    align-items:center;
    height:100vh;
}

.container{
    background:rgb(61, 58, 209);
    padding:20px;
    width:350px;
    border-radius:10px;
    box-shadow:0 0 10px rgba(0,0,0,0.2);
}

input{
    width:70%;
    padding:10px;
}

button{
    padding:10px;
    cursor:pointer;
}

ul{
    list-style:none;
    padding:0;
```

```
    margin-top:15px;
}

li{
    display:flex;
    justify-content:space-between;
    padding:8px;
    border-bottom:1px solid #ddd;
}

.completed{
    text-decoration:line-through;
    color:gray;
}
</style>
</head>

<body>

<div class="container">
<h2>To-Do List</h2>

<input type="text" id="taskInput" placeholder="Enter task">
<button id="addBtn">Add</button>
```

```
<ul id="taskList"></ul>

</div>


<script>

const input = document.getElementById("taskInput");

const addBtn = document.getElementById("addBtn");

const list = document.getElementById("taskList");


/* Add Task */

function addTask(){

    const text = input.value.trim();

    if(text===="") return;


    const li = document.createElement("li");

    li.innerHTML = `

        <span>${text}</span>

        <div>

            <button class="completeBtn">✔</button>

            <button class="deleteBtn">❌</button>

        </div>

    `;

    list.appendChild(li);

    input.value="";

}
```

```
/* Event Delegation */

list.addEventListener("click",(e)=>{



    if(e.target.classList.contains("deleteBtn")){

        e.target.closest("li").remove();

    }



    if(e.target.classList.contains("completeBtn")){

        e.target.closest("li").classList.toggle("completed");

    }



});



addBtn.addEventListener("click",addTask);

</script>



</body>

</html>
```

Output :-



Explanation :-

- This application dynamically adds tasks to the DOM without refreshing the page.
- Each task contains buttons to mark it complete or delete it.
- Event delegation is used by attaching one click listener to the task list instead of multiple listeners.
- DOM manipulation methods like createElement and appendChild are used to build UI dynamically.
- The code is modular by separating addTask logic and event handling.

Problem 4 :-

Develop a Mini Product Search Application using Vanilla JavaScript that displays products from a predefined array, filters them dynamically as the user types using DOM manipulation and event delegation, shows “No Results Found” when necessary, follows a modular structure, supports real-time filtering, and demonstrates basic state management.

Code :-

```
<!DOCTYPE html>
```

```html
<html>

<head>

<title>Mini Product Search</title>



<style>

body{

    font-family:Arial;

    background:linear-gradient(135deg,#74ebd5,#acb6e5);

    display:flex;

    justify-content:center;

    align-items:flex-start;

    height:100vh;

    padding-top:40px;

}



.container{

    background:white;

    width:400px;

    padding:20px;

    border-radius:12px;

    box-shadow:0 10px 25px rgba(0,0,0,0.2);

}



h2{

    text-align:center;
```

```
}


input{
    width:85%;
    padding:12px;
    margin-top:10px;
    border-radius:8px;
    border:1px solid #ddd;
    font-size:16px;
}


.product{
    background:#f7f7f7;
    padding:12px;
    margin-top:10px;
    border-radius:8px;
    display:flex;
    justify-content:space-between;
    transition:0.2s;
}


.product:hover{
    transform:scale(1.03);
    background:#e9f5ff;
}
```

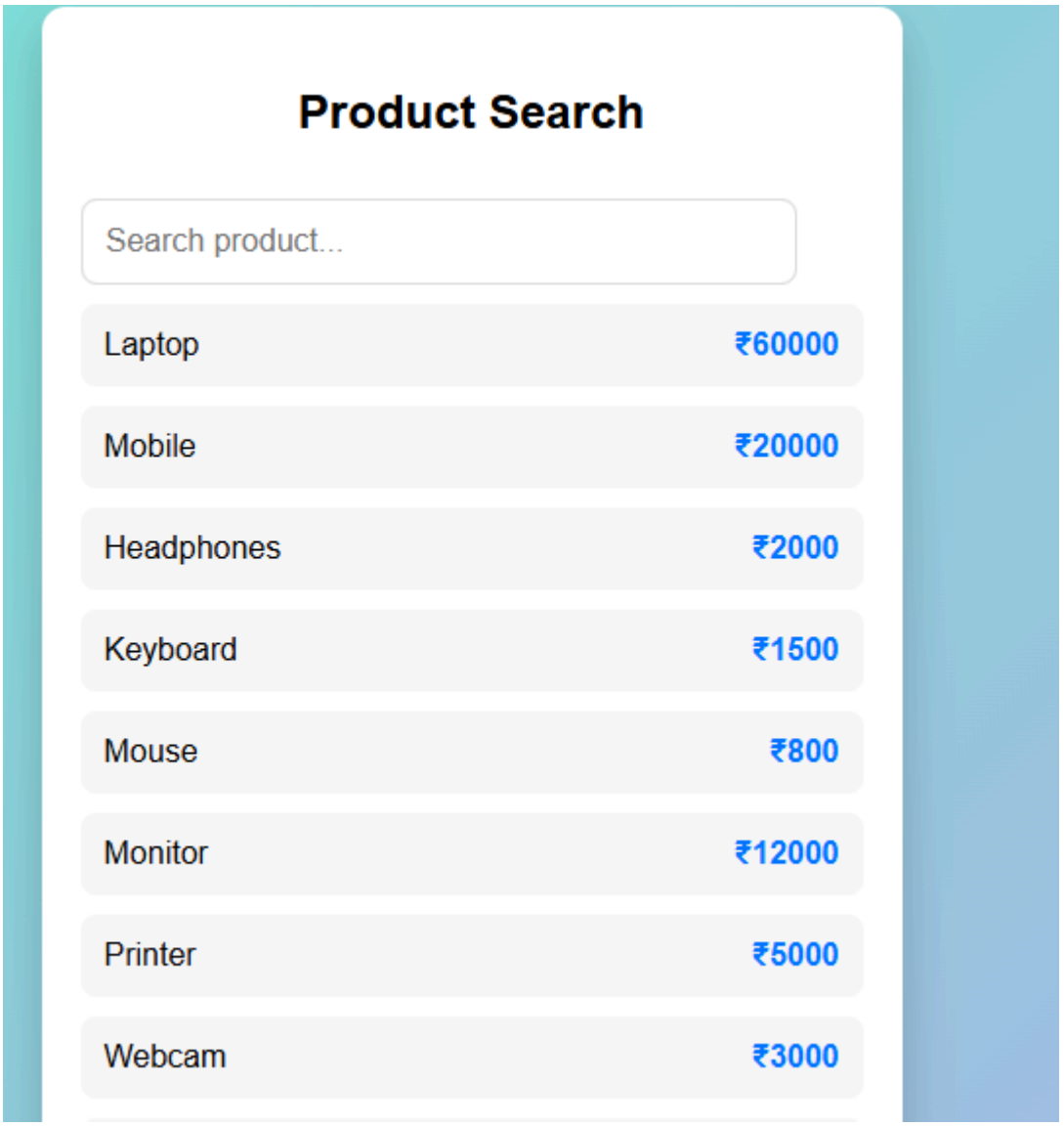
```
.price{
    font-weight:bold;
    color:#0077ff;
}

#noResult{
    text-align:center;
    margin-top:15px;
    color:red;
}
</style>
</head>

<body>

<div class="container">
<h2>Product Search</h2>

<input type="text" id="search" placeholder="Search product...">

<div id="productList"></div>
<div id="noResult"></div>
</div>
```

```
<script>


/* State (Product Array) */

const products = [

    {name:"Laptop",price:60000},

    {name:"Mobile",price:20000},

    {name:"Headphones",price:2000},

    {name:"Keyboard",price:1500},

    {name:"Mouse",price:800},

    {name:"Monitor",price:12000},

    {name:"Printer",price:5000},

    {name:"Webcam",price:3000},

    {name:"Speaker",price:2500},

    {name:"External Hard Drive",price:4000},

    {name:"USB Flash Drive",price:500},




];


const searchInput = document.getElementById("search");

const productList = document.getElementById("productList");

const noResult = document.getElementById("noResult");


/* Render Products */

function render(list){
```

```
    productList.innerHTML="";

    if(list.length===0){

        noResult.textContent="No Results Found";

        return;

    }

    noResult.textContent="";



    list.forEach(p=>{

        const div=document.createElement("div");

        div.className="product";

        div.innerHTML=`<span>${p.name}</span><span
class="price">₹${p.price}</span>`;

        productList.appendChild(div);

    });

}



/* Filter Function */

function filterProducts(){

    const value=searchInput.value.toLowerCase();

    const
filtered=products.filter(p=>p.name.toLowerCase().includes(value));

    render(filtered);

}



/* Event */

searchInput.addEventListener("input",filterProducts);
```

```
/* Initial render */
render(products);

</script>

</body>
</html>
```

Output :-



Explanation :-

- This application displays products from a predefined array and renders them dynamically using DOM manipulation.
- The search input listens to the input event to filter products in real time.
- A render function manages UI state by updating the product list.
- When no products match, a “No Results Found” message is displayed.
- This demonstrates basic state management and dynamic UI updates.